

WORLDLINE

User and Operations Manual

Branchable working realities for Omarchy · engine 1.2.0 · plugin 1.3.0 · 2026-09-20

1. Orientation

What WORLDLINE is, in one paragraph

WORLDLINE captures the working state of directories you nominate and calls that capture **PRIME**. It can fork PRIME into isolated **worlds**, run a coding agent inside each world where it cannot reach your real files, measure what each world actually produced, and commit exactly one world back into PRIME through a single atomic filesystem exchange that a formally proved kernel authorized. If the kernel refuses, or anything moved underneath it, PRIME is unchanged. `return` restores the previous reality through the same mechanism.

What it is not

- **Not a replacement for version control.** Use Git as usual. WORLDLINE tracks the whole working state around it, including uncommitted files and generated artifacts, and (since 1.2.0) a commit an agent makes inside its world lands in your repository when that world collapses.
- **Not a backup system.** Archived generations are retained until you `prune` them; retention is a disk policy, not a backup policy, and WORLDLINE never copies anything off the machine except the receipt anchor ledger you configure.
- **Not a process freezer.** Files, repositories, configuration, and evidence are captured; running processes, terminals, and window layout are observed and reported, not restored.

Typographic conventions

Convention	Meaning
fixed width	Text you type, a file path, a field name, or output
UPPERCASE	A value you supply, such as <code>WORLD</code> or <code>PATH:LINE</code>
[...]	Optional
A \ B	Choose one

2. Safety notice: what WORLDLINE changes

Two operations move your real files: registering a managed root, and committing a collapse or return.

Everything else is read-only or contained inside a world.

Operation	Effect on your files	Confirmation
<code>worldline init</code> , <code>worldline root add</code>	Moves the directory into a WORLDFLINE generation on the same filesystem and replaces the original path with a symlink	Printed root list, then a prompt or <code>--yes</code> ; <code>--dry-run</code> prints only
<code>worldline root remove</code>	Materializes the current payload back at the original path, then removes the link	Prompt or <code>--yes</code> ; <code>--dry-run</code>
<code>worldline collapse</code> <code>WORLD</code>	Replaces the live mapping so PRIME becomes the candidate applied to current state	Facts screen, then prompt or <code>--yes</code> ; or <code>--prepare</code> then <code>transaction commit ID</code> <code>--yes</code>
<code>worldline return</code> <code>[WORLD]</code>	Materializes a checkpoint into a new live generation	Same as collapse
<code>worldline prune</code>	Deletes payload directories of finished worlds nothing refers to	Printed plan, then prompt or <code>--yes</code> ; <code>--dry-run</code>
<code>worldline fork</code> , <code>race</code> , <code>ghost run</code>	Nothing outside the world's own overlay; spends external model quota	None; starting is the decision
<code>worldline simulate</code>	Nothing on the host filesystem; the command runs in a disposable system future that shares the host network	None
Everything else	Read-only	None

- **Installation does not touch your work.** The installer never registers, moves, or migrates a working directory.
- **Non-interactive callers must be explicit.** Without a terminal, a mutation without `--yes` fails with `CONFIRMATION_REQUIRED`.
- **Refusal is normal.** A collapse that would overwrite a change you made in the meantime refuses with `CONFLICT` and names the paths. Nothing is written on a refusal.
- **Agents run with their own permission prompts disabled**, because the sandbox is the boundary for your files. It is not a boundary for the network unless you set `network.policy` (§5.2): under the default `shared` policy an agent can reach anything the host can.

3. Requirements

Requirement	Used for	On this machine
Linux with overlayfs and user namespaces, bubblewrap	World isolation	Present
systemd user session	Daemon and job supervision	Running
Python 3.12+ with cryptography	Runtime, receipt anchors	3.14
GNAT and gnatprove	Building and proving the kernel (installation only)	~/opt/gnat
Git	repo root capture	Present
attest (optional)	Independent proved verification of the anchor ledger	Present
Hyprrland and the Omarchy shell	Desktop surfaces	1 monitor
Btrfs, CRIU	Snapshot and checkpoint backends	Unavailable; reported honestly

All managed roots must live on the same filesystem as the WORLDLINE data store.

4. Installation

4.1 Running the installer

```
cd ~/Projects/worldline
./install.sh
```

The installer refuses to install anything until the whole gate passes: it builds the Ada library, runs the Ada behaviour and fuzz executables, runs the runtime test suite (94 tests), runs the proof gate (130 checks, all proved, nothing assumed), and independently re-verifies the proof manifest. It refuses to restart a daemon that is supervising running agent jobs (`WORLDLINE_FORCE=1` overrides). It then writes a backup, stages the runtime, library, launchers, service unit, and the plugin checkout, patches the shell bar and key bindings idempotently, restarts the daemon, reloads Hyprrland, and restarts the shell (`WORLDLINE_NO_SHELL_RESTART=1` defers that).

4.2 What the installer writes

Path	Content	Behaviour
~/local/lib/worldline	Runtime package, <code>libworldline_core.so</code> , <code>proof-manifest.json</code> , C header	Replaced atomically

Path	Content	Behaviour
~/.local/bin/ worldline, ~/.local/bin/worldlined	Launchers	Overwritten
~/.config/omarchy/ plugins/ khephri.worldline	Desktop plugin, a git checkout of ../worldline-omarchy	Fast-forwarded; a diverged checkout is refused, never overwritten
~/.config/systemd/ user/ worldlined.service	User service unit	Enabled and restarted
~/.config/omarchy/ shell.json, ~/.config/hypr/binding s.lua	One bar entry, one marked binding block	Backed up, patched idempotently
~/.local/state/ worldline/install- backups/<stamp>-<pid>	Previous runtime, library, launchers, unit, shell config, plugin commit	One per run; see §4.4

4.3 Verifying the installation

```
systemctl --user is-active worldlined.service      # active
worldline status --json | jq .daemon              # version, RUNNING
worldline doctor --json                          # every integrity row OK;
anchor, storeUsage, networkPolicy, limits
worldline adapters --json                        # your agents AVAILABLE, or an
UNAVAILABLE reason
bash ~/.claude/skills/worldline/scripts/health_check.sh  # 47 assertions, zero
quota, host untouched
```

Before any root is registered, status reports `prime: null` and empty `worlds` and `jobs`. That is the correct post-installation state.

4.4 Rollback

```
B=~/.local/state/worldline/install-backups/<stamp>
systemctl --user stop worldlined.service
rm -rf ~/.local/lib/worldline && cp -a "$B/lib-worldline" ~/.local/lib/worldline
install -m 0755 "$B/worldline" "$B/worldlined" ~/.local/bin/
install -m 0644 "$B/worldlined.service" ~/.config/systemd/user/worldlined.service
install -m 0600 "$B/shell.json" ~/.config/omarchy/shell.json
install -m 0600 "$B/bindings.lua" ~/.config/hypr/bindings.lua
git -C ~/.config/omarchy/plugins/khephri.worldline checkout --quiet "$(cat
"$B/plugin-commit")"
systemctl --user daemon-reload && systemctl --user start worldlined.service
```

The store is not part of the backup and is not touched by a rollback. Store format changes are forward-

only migrations (§10.4); an older runtime opens a newer store only if no migration lies between them, and refuses by name otherwise.

5. First-time setup

5.1 Registering managed roots

```
worldline init ~/Projects/myapp
worldline init --dry-run ~/Projects/myapp      # prints the exact effect, changes
nothing
worldline root list --json
worldline root add ~/.config/myapp
worldline root remove ~/.config/myapp
```

- Kind defaults to `repo` for a Git working tree, `config` under your XDG configuration directory, and `filesystem` otherwise; override with `--kind`.
- The primary root is the first registered root unless `--primary PATH` says otherwise. It is the agent's working directory and the home of `.worldline.json`.
- Registration moves the directory into a `WORLDLINE` generation on the same filesystem and leaves a symlink at the original path. Absolute paths keep working.
- Refused: symlinks, non-directories, overlapping roots, roots on another filesystem, paths that overlap `WORLDLINE`'s own state, and any root-set change while a transaction is open or a non-terminal world exists (`ROOT_SET_BUSY`).

5.2 Global configuration

`~/.config/worldline/config.json` is owner-only (`UNSAFE_CONFIG` otherwise). It has four required fields and three optional blocks.

```
{
  "schemaVersion": 1,
  "readonlyHomePaths": ["/home/you/.local/bin", "/home/you/.local/share/mise"],
  "agentCommands": {
    "my-tool": {
      "argv": ["/usr/bin/my-tool", "--json", "--cwd", "{workspace}", "--mission",
        "{missionFile}"],
      "credentialMounts": [{"source": "/home/you/.config/my-tool/auth.json",
        "target": "/home/you/.config/my-tool/auth.json"}],
      "eventFormat": "jsonl",
      "networkHosts": ["api.my-provider.example"]
    }
  },
  "ghosts": {"enabled": false, "agent": null},
  "limits": {"defaultTimeoutSeconds": 1800},
  "network": {"policy": "allowlist", "allow": []},
  "anchor": {"exportPath": "/home/you/Dropbox/worldline-anchor"}
}
```

Field	Meaning
<code>readonlyHomePaths</code>	Paths projected read-only into every world (toolchains). Must not overlap WORLDLINE storage.
<code>agentCommands</code>	Generic adapters: exactly <code>argv</code> , <code>credentialMounts</code> , <code>eventFormat</code> , optionally <code>networkHosts</code> . Placeholders <code>{workspace}</code> , <code>{missionFile}</code> , <code>{worldState}</code> ; no shell.
<code>ghosts</code>	Speculative-world opt-in. Enabling requires an agent name and spends quota on every checkpoint.
<code>limits.defaultTimeoutSeconds</code>	<code>null</code> or a positive integer applied to every world unless <code>--timeout</code> overrides.
<code>network.policy</code>	<code>shared</code> : worlds use the host network (egress not contained). <code>allowlist</code> : each world gets an empty network namespace and one door, a daemon-side proxy that reaches only the agent's provider hosts plus <code>network.allow</code> ; every refused host is recorded on the world. <code>none</code> : no door. Checks and services get no network under <code>allowlist</code> or <code>none</code> .
<code>network.allow</code>	Extra hosts for <code>allowlist</code> ; a leading dot allows a domain (<code>.example.org</code>). Claude Code's remote MCP connectors (<code>mcp-proxy.anthropic.com</code>) are refused by default; add them here if a mission needs them.
<code>anchor.exportPath</code>	Absolute directory outside WORLDLINE storage that receives a copy of the signed receipt ledger after every commit (§9.3).
<code>adapterOptions.<builtin>.argv</code>	Extra <code>argv</code> inserted before the mission for a builtin adapter — the way to choose a model or reasoning effort for worlds without editing your own tool config, e.g. <code>codex ["-m", "gpt-5.6-luna", "-c", "model_reasoning_effort=max"]</code> . <code>worldline adapters</code> shows the resulting <code>argv</code> .

Builtin adapters on this machine: **claude** (runs with `--verbose`, hooks disabled in-world), **codex**, **omp** (a private per-world copy of its database), **pi** (UNAVAILABLE until `pi login`). `worldline adapters` says which are usable and why not.

5.3 Project configuration

`.worldline.json` in the primary root declares how a world is measured. Without it, worlds still fork and finalize, but evidence is **UNASSESSED** and risk cannot fall below **MEDIUM**.

```
{
  "schemaVersion": 1,
  "generated": [{"root": "/home/you/Projects/myapp", "glob": "dist/*"}],
  "checks": [
    {"id": "build", "kind": "build", "argv": ["/usr/bin/make", "build"],
    "required": true, "format": "exit"},
    {"id": "tests", "kind": "tests", "argv": ["/usr/bin/make", "test"], "required":
    true, "format": "junit", "result": "build/junit.xml", "covers": ["src/*"]},
    {"id": "proofs", "kind": "proofs", "argv": ["/usr/bin/prove.sh"], "required": false,
    "format": "gnatprove"},
    {"id": "bench", "kind": "benchmark", "argv": ["/usr/bin/bench.sh"], "required": false,
    "format": "worldline-benchmark-v1", "result": "build/bench.json"}
  ],
  "services": [
    {"id": "api", "argv": ["/usr/bin/myapp", "serve"], "cwd": ".", "env":
    {"MYAPP_PORT": "8080"}, "healthArgv": ["/usr/bin/curl", "-fsS",
    "http://127.0.0.1:8080/health"], "restart": "on-failure"}
  ]
}
```

generated globs mark artifacts a project writes into its own root while it is live; they are excluded from dirtiness and are exactly what `return` preserves when it restores a checkpoint that was live (§7.5). A benchmark result supplies `metric`, `unit`, `baseline`, `candidate`, and `direction`; **WORLDLINE** never invents a performance figure.

6. Everyday workflow

6.1 Fork one world

```
worldline fork alpha --mission-text 'Add retry to the upload path' -- codex
worldline fork alpha --mission mission.md --wait --timeout 900 -- claude
echo 'Refactor the parser' | worldline fork beta -- codex
```

- The alias is yours; identity is the content hash. Aliases are unique, trimmed, no slash or NUL.
- The agent follows `--fork`. `fork` refuses `ADAPTER_AUTH_UNAVAILABLE` before creating anything when the agent's credentials are missing.
- Mission precedence: `--mission FILE`, `--mission-text`, piped stdin, `<primary-root>/mission.md`.
- `--timeout SECONDS` (or `limits.defaultTimeoutSeconds`) stops a runaway agent: the world ends **DEGRADED**, its agent check reads **FAIL** · **TIMEOUT**: ..., project checks are **UNASSESSED**, the job is **TIMED_OUT**, and the partial work is still inspectable.
- Your files are untouched. The world writes to its own overlay.

6.2 Race three agents

```
worldline race --name retry --mission-text 'Three designs for the scheduler' \
  --agent codex --agent claude --agent codex --detach --timeout 1200
```

Exactly three `--agent` values, three siblings from one frozen checkpoint, lanes `alpha`, `beta`, `gamma` (prefixed by `--name`). They are scored only after each reaches a terminal state.

6.3 Watch, list, and inspect

```
worldline status          worldline list          worldline show beta
worldline graph           worldline watch         worldline inspect beta
worldline why src/app.py:42
```

`show` reports what was observed: delta counts, each check with its status, the agent check's network block (policy, connections, refused hosts), complexity, risk, conflicts, contamination, repository facts for git roots (head, branch, dirty), and `payload_pruned`. Where a capability was unavailable, the reason is printed instead of a guess.

6.4 Collapse the world you want

```
worldline collapse beta          # facts screen, then a prompt
worldline collapse beta --prepare --json  # stage, print facts, leave it
PREPARED
worldline transaction list
worldline transaction commit TRANSACTION_ID --yes
worldline transaction abort TRANSACTION_ID
```

`--prepare` is what the cockpit uses: the facts (decision, delta, conflicts, contamination) are computed once, shown, and the commit re-verifies `PRIME` against the prepared `beforeRoot`. A change in between is refused `PRIME_CHANGED_AFTER_PREPARE`. Only a `VALID` world can collapse; a refusal names its decision code and writes nothing. For a git root, a commit the agent made inside its world is part of the delta and lands in the live repository.

6.5 Return to an earlier reality

```
worldline return          # to the checkpoint that preceded current PRIME
worldline return WORLD    # to a specific archived checkpoint
worldline return --prepare
```

Return restores the state a checkpoint had at the instant it was displaced. A checkpoint that was live and accumulated generated artifacts is verified against the receipt or reconcile checkpoint that displaced it; one that changed after it left reality is refused `PAYLOAD_INTEGRITY_FAILED` with the reason. A pruned checkpoint refuses `PAYLOAD_PRUNED`.

6.6 Ask why a line exists

`worldline why PATH:LINE` walks the causal chain (mission, agent events, tool calls, receipts) to

the world and event that last touched that line. If the adapter supplied only file-level provenance, **why** says so rather than inventing a line.

6.7 Audition a system change

`worldline simulate COMMAND...` runs the exact argv as namespace root in a disposable system future over `/usr`, `/etc`, and `/var`, runs declared health checks, and reports the diff. Nothing on the host filesystem changes. The future shares the host network, so a network side effect is real. System futures cannot collapse on ext4 (`SYSTEM_ROOT_COLLAPSE_UNSUPPORTED`).

6.8 Ghost worlds

`worldline ghost enable --agent claude` makes every new PRIME checkpoint fork four low-priority speculative worlds (security-refactor, performance, remove-dependency, simplification). Each one calls a model with your credentials: **ghosts spend model quota**. `ghost status` shows a recommendation only when objective evidence improved; `ghost disable` stops them. A ghost can never collapse itself.

6.9 Cancel a running world

`worldline cancel WORLD` stops the agent's unit. The agent check reads `FAIL` · `USER_CANCELLED`, project checks are `UNASSESSED`, the job is `CANCELLED`, and the partial work is inspectable.

7. Reading results honestly

7.1 States

`MUTABLE` (running), `FINALIZING`, `VALID`, `DEGRADED` (a required check failed, the agent was stopped, or supervision was lost), `DEAD` (never finished; the reason is on `evidence.supervision`), `ARCHIVED`, `COLLAPSED`. Only `VALID` can collapse.

7.2 Evidence vocabulary

`PASS`, `FAIL`, `UNASSESSED` (no evidence configured or none ran), `UNAVAILABLE` (a capability was missing, with a reason), `STALE` (the daemon stopped publishing). The cockpit renders an optional-check failure as `PASS` · `N GAP`, never as `FAIL`.

7.3 Risk and complexity

Derived labels whose inputs are next to them: checks with `required` and `status`, conflicts, contamination, delta, and state. `scripts/pick_candidate.py` recomputes them; it hard-filters PRIME generations, anything not `VALID`, conflicts, contamination, and failed required

checks, and refuses to recommend when evidence is UNASSESSED.

7.4 Network evidence

Under `allowlist`, the agent check's `network.refused` lists every host the agent tried to reach outside its allowlist, with counts. That is the policy working; the host name tells you what the agent wanted (on this machine a claudeworld refused `github.com`, Datadog telemetry, and the remote MCP proxy while completing its mission).

7.5 Proof claims

`invariantPreservation`: **PROVED** in a receipt means the installed proof manifest still matches the running library. The proved kernel supplies the equality verdict and the state-machine rules; the runtime decides what is compared and performs the exchange. The C/Python/QML boundary and the OS are outside the proof, and the receipt says so under `boundary.notProved`.

8. The desktop cockpit

The bar widget shows a globe; when a world runs it carries a badge. **SUPER+SHIFT+W** opens the fork surface, **SUPER+CTRL+W** the multiverse, **SUPER+ALT+RIGHT** runs `switch --next`, **Escape** closes. Inside: **F** fork, **M** roots, **C** collapse (prepare → review → confirm → commit), **R** return, **X** cancel, **L** load the agent log, **?** keys, **0** resets the graph zoom.

The diagnostics card is `worldline doctor` rendered: managed roots, store, receipts, recovery, supervision, **anchor** (entries, attest verdict, external copy), **store usage**, **network** policy, **timeout** default, and capabilities. A generation wider than the viewport is fitted automatically until you zoom or pan.

9. Operating limits and hygiene

9.1 Timeouts

Every real run should carry one. `--timeout` on `fork` and `race`, or the config default. A timed-out world is honest: **DEGRADED**, reason on the agent check, checks **UNASSESSED**.

9.2 Prune

```
worldline prune --dry-run
worldline prune --older-than 14 --keep 5
worldline prune --logs --yes
```

Every fork materializes a full checkpoint and every finished world keeps its payload, so the store only grows. `prune` deletes the payload directories of finished worlds that nothing living refers to and marks them `payload_pruned`. Never touched: **PRIME**, the checkpoint `return` goes back to, running

worlds, and any directory still referenced by an unpruned world or a live root. Records, receipts, and causal events stay; a `prune` event is recorded per world. A pruned world can no longer be inspected, shelled into, or returned to. `doctor .storeUsage` shows bytes by area.

9.3 Anchored receipts

Every committed receipt is appended to an Ed25519-signed, hash-chained ledger in the Custos format (`~/.local/state/worldline/anchor.tsv`, key in `~/.config/worldline/anchor/`). Verify it three ways:

```
worldline anchor                # local replay, attest verdict, external
comparison
worldline log --verify          # receipts + anchor
attest verify-custos ~/.local/state/worldline/anchor.tsv
~/.config/worldline/anchor/public.hex
```

`attest` replays the chain and every signature with the SPARK-proved implementation, using none of WORLDLINE's code. `anchor.exportPath` mirrors the ledger somewhere the store's writer does not control; the verdicts are `MATCH`, `EXPORT_BEHIND`, `ROLLED_BACK` (the local ledger is shorter than the copy), `MISMATCH`, or `UNCONFIGURED`. What this does not defend against: an attacker who also holds the signing key and can reach the export location.

9.4 Network policy

See §5.2. `allowlist` is the recommended setting for missions that touch untrusted content. `simulate` always shares the host network; the manual says so because the containment claim must not be overstated.

10. Reference

10.1 Command surface

```
status list show WORLD graph log [--verify] why PATH:LINE inspect WORLD
watch
init ROOT... [--primary P] [--kind repo|config|filesystem] [--dry-run] [--yes]
root add ROOT... [--dry-run] [--yes] | root remove ROOT [--dry-run] [--yes] | root
list
fork NAME [--mission F | --mission-text T] [--wait] [--timeout S] -- AGENT
race [MISSION_FILE] [--mission-text T] --agent A --agent B --agent C [--name N] [--
detach] [--timeout S]
collapse WORLD [--yes | --prepare] return [WORLD] [--yes | --prepare]
transaction list | show ID | commit ID --yes | abort ID
cancel WORLD prune [--older-than D] [--keep N] [--logs] [--dry-run] [--yes]
anchor
simulate COMMAND... doctor [--refresh] adapters shell WORLD switch [--
next|--previous|WORLD]
ghost enable --agent A | ghost disable | ghost status | ghost run OBJECTIVE [--
wait]
```

Every command accepts `--json`; exit 0 success, 1 a named error (`worldline: CODE: message`), 130 interrupted.

10.2 Error codes worth knowing

DAEMON_UNAVAILABLE, NO_PRIME, NO_MISSION, CONFIRMATION_REQUIRED, CONFLICT, INVALID_CANDIDATE, PRIME_CHANGED_DURING_CAPTURE, PRIME_CHANGED_AFTER_PREPARE, STAGED_ROOT_MISMATCH, PAYLOAD_INTEGRITY_FAILED, PAYLOAD_PRUNED, PRUNE_BLOCKED, ROOT_SET_BUSY, RETURN_POINT_INCOMPLETE, ADAPTER_UNAVAILABLE, ADAPTER_AUTH_UNAVAILABLE, TIMEOUT, USER_CANCELLED, DISK_FULL, STORAGE_ERROR, NETGUARD_UNAVAILABLE, UNSUPPORTED_SCHEMA, GHOSTS_DISABLED, SYSTEM_ROOT_COLLAPSE_UNSUPPORTED.

10.3 Status document and daemon protocol

`status.json` under `$XDG_RUNTIME_DIR/worldline` has exactly nine fields: `schemaVersion`, `daemon`, `prime`, `activeWorld`, `worlds`, `jobs`, `capabilities`, `lastReceipt`, `ghostRecommendation`. A world's `delta.files` is capped at 200 entries with `truncated` and `total`; `show` returns everything. The daemon speaks NDJSON over an owner-only Unix socket with a peer-credential check per connection.

10.4 Store schema

The SQLite store carries `PRAGMA user_version = 2`. Migrations are forward-only and recorded in `meta.schemaMigrations`; a store newer than the runtime is refused `UNSUPPORTED_SCHEMA`.

10.5 Uninstalling

`worldline root remove` every root **first** (it materializes your directories back), then disable the unit, remove `~/.local/lib/worldline`, the launchers, the plugin checkout, the `shell.json` entry, and the bindings block. The store under `~/.local/share/worldline` and `~/.local/state/worldline` can then be deleted.